

Personalized AI-Based Investment Advisor

CSE455 – ARTIFICIAL INTELLIGENCE

Offered by the

Department of Computer Science and Engineering

School of Engineering and Sciences

SUBMITTED BY

MARAM HASINI SAI (AP23110010502)



SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[APRIL, 2026]

CERTIFICATE

This is to certify that the project work entitled “[Path Navigation of Autonomous Agents using Artificial Intelligence]” is a Bonafide record of project work carried out by the following students:

- Ms. M. Hasini Sai (AP23110010502)
- Mr. P. Gunavanth Reddy (AP23110010505)
- Ms. Anjali Desai (AP23110010509)
- Mr. Siddi Sathvik Kuruba (AP23110010516)
- Ms. Shaik Saniya Haseen (AP23110010517)

from the Department of Computer Science and Engineering, SRM University-AP. The students conducted this project work under my supervision during the period January 2026 to May 2026. It is further certified that, to the best of my knowledge, this project has not previously formed the basis for the award of any degree or any similar title to this or any other candidate. This is also to certify that the project work represents the teamwork of the candidates.

Station: Mangalagiri

Date:

Dr. Subhankar Ghatak

Assistant Professor

Department of Computer Science & Engineering

SRM University-AP

Andhra Pradesh.

TABLE OF CONTENTS

S. NO.	TITLE	PAGE NO.
1	Abstract	4
2	Introduction	6
3	Aim / Objective	8
4	System Requirements	8
5	System Architecture Overview	9
6	Methodology / Algorithm	11
7	Program Code	14
8	Output / Observation	15
9	Limitations and Future Work	17
10	Conclusion	19
11	References	20

ABSTRACT

SmartWealth is a Python-based intelligent financial advisory application built on the Streamlit framework, conceived to place credible, data-grounded investment guidance directly in the hands of everyday users. At a time when personal finance decisions carry enormous long-term consequences yet sophisticated advisory tools remain largely confined to institutional settings, SmartWealth offers a self-contained alternative — one that thinks through a user's financial position and translates that thinking into a concrete, actionable portfolio strategy.

The application gathers four inputs from the user — **age, income, risk appetite**, and **investment goal** — and uses these to construct a financial profile that drives every downstream decision. Age determines the appropriate investment horizon and tolerance for short-term volatility. Income, displayed in Indian Rupees to serve the platform's primary target audience of Indian investors, sets the scale of capital available for allocation. Risk appetite captures the user's psychological comfort with uncertainty — a dimension that pure numerical data cannot detect on its own. And the investment goal anchors the entire exercise to something meaningful: whether the user is saving for retirement, building an emergency fund, financing education, or growing long-term wealth shapes the character of the strategy the system recommends.

Under the hood, SmartWealth draws on three core analytical capabilities. **Rule-based AI logic, optionally powered by Prolog**, evaluates the user's profile against a structured library of investment principles to determine which strategy classification applies — conservative, balanced, or growth-oriented — in a manner that is fully traceable. **Live market data retrieved through yFinance** ensures that portfolio suggestions are rooted in current price movements and recent market behaviour rather than static assumptions. An **interactive chatbot interface** then gives users the ability to ask questions and explore their recommendation conversationally, turning a one-way output into a genuine dialogue. Session data is retained in a local **SQLite database**, preserving continuity between interactions without requiring cloud infrastructure or external servers.

The user experience is delivered through a clean, dark-themed financial dashboard whose visual language was deliberately chosen to feel credible and professional rather than toy-like. **Plotly-powered interactive charts** render portfolio allocation breakdowns, historical price trends, and market sentiment indicators in a format that communicates insight at a glance. The combination of a polished interface, intelligent back-end logic, and real-time data integration makes SmartWealth a working proof-of-concept for what accessible, AI-driven financial advisory can look like when built with care.

INTRODUCTION

For most of recorded financial history, the ability to receive structured, personalised investment advice has been tied to one's ability to pay for it. Licensed financial advisors charge fees that place their services out of reach for large segments of the population; robo-advisors, while more accessible, typically operate as black boxes that tell users what to do without explaining why. The result is a landscape in which the individuals who most need guidance — those without existing wealth, financial literacy, or professional connections — are precisely the ones least likely to receive it. SmartWealth was built to push back against that reality.

The project's central ambition is straightforward: to construct an AI-powered advisory platform that studies a user's financial circumstances honestly, applies principled reasoning to those circumstances, and returns a recommendation that is both personalised and explainable. Unlike systems that generate outputs users simply have to trust, SmartWealth's rule-based architecture means every strategic conclusion can be traced back to a specific logical condition. A user who wants to understand why they were steered toward a conservative allocation rather than an aggressive one can follow that reasoning — it is not buried inside a neural network's weight matrix but expressed in auditable rules that reflect real investment principles.

The platform was designed with Indian retail investors specifically in mind. Income figures are handled and displayed in Indian Rupees (INR), investment goals reflect priorities common in the Indian financial context, and the overall system was calibrated to be deployable either as a local desktop application or via Streamlit Cloud — making it accessible without requiring users to install complex software or manage technical infrastructure. This flexibility was a deliberate design choice: a tool that solves a real problem but demands technical expertise to run has not truly solved the accessibility problem.

Three specific real-world challenges shaped the design of the system in ways that distinguish it from simpler financial calculators. The first is the challenge of interpreting subjective risk tolerance. Risk is not a number — it is a psychological disposition that varies enormously between individuals who might otherwise appear financially similar. SmartWealth captures this through a dedicated risk appetite input and factors it explicitly into the strategy classification rather than

inferring risk tolerance indirectly from age or income alone. The second challenge is mapping diverse financial goals to concrete strategies. A user saving for retirement in twenty years requires a fundamentally different portfolio structure than one building a short-term emergency fund, even if their income and age are identical. The system's rule engine handles this mapping explicitly, ensuring that the goal drives the outcome rather than being treated as an afterthought. The third challenge is presenting live market sentiment in an accessible way. Raw sentiment scores mean nothing to most users; SmartWealth translates them into visual indicators and plain-language chatbot responses that communicate market mood without requiring any prior knowledge of NLP or financial analysis.

Taken together, these design decisions produce a system that occupies a genuinely useful position in the financial technology landscape — more intelligent and personalised than a generic investment calculator, more transparent and accessible than a conventional robo-advisor, and more grounded in real market conditions than a purely rule-based expert system operating in isolation. The sections that follow document the architecture, data flow, technology choices, and output design that bring this vision to life, as well as an honest account of where the system currently falls short and where it is headed next.

AIM / OBJECTIVE

The primary objectives of this project are:

- Provide personalized investment strategy recommendations based on user profile inputs (age, income, goal, risk appetite).
- Implement AI-based reasoning using Prolog inference rules, with a Python fallback for portability.
- Integrate real-time market data for live stock/index price sentiment display.
- Offer an interactive financial chatbot to answer basic investment queries.
- Persist all recommendation history in a local SQLite database for future reference and analysis.
- Deliver a professional, intuitive user interface optimized for non-technical end users.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Any modern dual-core CPU (Intel i3 or equivalent)
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: 500 MB free space (for Python environment and SQLite DB)
- Internet Connection: Required for live market data via yFinance API

Software Requirements

- Python 3.8 or above
- pip (Python package manager)
- Optional: SWI-Prolog (for Prolog-based reasoning engine)
- Operating System: Windows, macOS, or Linux

SYSTEM ARCHITECTURE OVERVIEW

The system is built around a layered, modular architecture. Below is a description of each layer and the data flow between components as illustrated in the Project Flow diagram.

Layer 1: User Interface

Technology: Streamlit Dashboard

The user interacts with the system through a Streamlit-powered web application. The interface collects three primary inputs:

- Age — to tailor risk tolerance and investment horizon.
- Income — to determine investable surplus and portfolio scale.
- Goal — the user's financial objective (e.g., retirement, wealth growth, education fund).

These inputs are forwarded to the Advisor Bridge (`advisor.py`) for further processing.

Layer 2: Advisor Bridge (`advisor.py`)

The `advisor.py` module acts as the middleware between the UI and the AI Core. It validates user inputs, formats them appropriately, and routes them to the correct AI subsystem components. It also aggregates outputs from all AI Core components and returns the final recommendation to the UI.

Layer 3: AI Core

The AI Core is the intelligence hub of the system. It consists of four tightly integrated subsystems:

- **Expert System Brain — Prolog Logic**

The Expert System uses Prolog-based logic to apply domain-specific investment rules. Based on the user's age, income, and goal, Prolog rules check the investment strategy logic and determine

a suitable portfolio profile (e.g., aggressive, moderate, conservative). The expert system receives strategy signals from the Market API and refines recommendations accordingly.

- Input: User profile (age, income, goal) + market strategy signal.
- Output: Investment strategy recommendation saved to the database.

- **Market API — yFinance**

The Market API component integrates with the yFinance library to retrieve live and historical stock market data. This includes:

- Real-time stock prices of relevant instruments.
- Historical price trends for analysis.
- Financial news headlines associated with targeted securities.

The Market API acts as the data backbone of the AI Core, providing factual market signals to both the Expert System Brain (for strategy checking) and the NLP Sentiment Engine (for news analysis).

- **NLP Sentiment Engine — TextBlob**

The NLP Sentiment Engine uses the TextBlob library to perform sentiment analysis on the financial news headlines fetched by the Market API. The sentiment scores (positive, negative, neutral) derived from news analysis are stored in the database and influence the final investment recommendations.

- Input: Financial news headlines from Market API.
- Processing: TextBlob-based polarity and subjectivity scoring.
- Output: Sentiment scores saved to SQLite database.

- **Database — SQLite**

A lightweight SQLite database serves as the central data store for the system. All outputs from the Expert System Brain and the NLP Sentiment Engine flow into the database. The database supports:

- Persistent storage of user profiles and recommendation history.
- Storage of market sentiment scores derived from news analysis.
- Query access for the AI Financial Chatbot to respond to user questions.

METHODOLOGY

Step	From	To	Data / Action
1	User Interface	Advisor Bridge	Age, Income, Goal submitted by user
2	Advisor Bridge	AI Core	Validated user profile forwarded
3	AI Core	Market API (yFinance)	Request stock prices & news headlines
4	Market API	Expert System Brain	Market strategy signal (Check Logic)
5	Market API	NLP Sentiment Engine	News headlines for analysis
6	NLP Sentiment Engine	Database (SQLite)	Analyzed headline sentiments saved
7	Expert System Brain	Database (SQLite)	Investment recommendations saved
8	Database	Dynamic Charts & AI Report	Combined data → visual output
9	Database	AI Financial Chatbot	Data queried for user questions

Step 1 — User Interface → Advisor Bridge Everything starts with the user sitting at the Streamlit dashboard. Three questions are posed: How old are you? What is your income? What are you trying to achieve financially? These answers — simple on the surface — carry significant weight. They define who the investor is, how much risk they can reasonably absorb, and what success looks like for them personally. The moment the user hits submit, those three values begin their journey through the system toward the Advisor Bridge.

Step 2 — Advisor Bridge → AI Core Think of the Advisor Bridge as a checkpoint. It does not perform any analysis itself — its job is to make sure the data arriving from the dashboard is clean, complete, and properly structured before anything intelligent is done with it. Incomplete entries are

flagged, formats are standardised, and the verified profile is then handed off to the AI Core in a form that every downstream subsystem can work with reliably.

Step 3 — AI Core → Market API (yFinance) The AI Core cannot recommend anything in isolation — it needs to know what the market is actually doing right now. So it reaches out to yFinance and pulls in a snapshot of the financial world: current stock prices, price movement history, and the headlines dominating financial news at this moment. This step is what separates a static, rule-only system from one that responds to real-world conditions. The recommendation will reflect today's market, not last month's.

Step 4 — Market API → Expert System Brain Stock prices tell a story — and the Market API reads that story into a strategic signal, essentially a summary of whether conditions currently favour risk-taking or caution. That signal travels to the Expert System Brain, which holds a library of investment rules coded in Prolog. Here, the user's personal profile meets the reality of the market. The Prolog engine works through its rule set, identifies which conditions are satisfied, and derives a logical investment strategy — conservative, balanced, or growth-focused — backed by transparent reasoning at every step.

Step 5 — Market API → NLP Sentiment Engine Numbers alone do not tell the full story of a market. Headlines do. While the Expert System Brain processes price signals, the same batch of news headlines fetched from yFinance is sent sideways to the NLP

Sentiment Engine. TextBlob reads each headline and scores it — is the tone hopeful or fearful? Confident or uncertain? This sentiment reading adds a human dimension to the analysis, capturing the collective mood of the financial world at this point in time, something that raw stock figures simply cannot express.

Step 6 — NLP Sentiment Engine → Database (SQLite) Once TextBlob finishes scoring the headlines, those sentiment values need a home. They are written into the SQLite database with

timestamps and associated context. Saving them here means they are not thrown away after a single use — they can later inform the visual reports, answer chatbot queries, and potentially reveal how market sentiment has shifted across multiple sessions over time.

Step 7 — Expert System Brain → Database (SQLite) With its logical evaluation complete, the Expert System Brain commits its conclusions to the database. The recommended strategy, the reasoning chain behind it, and the associated portfolio guidance are all stored alongside the user's profile. This record becomes the authoritative output of the AI Core — a single reliable source that both the reporting engine and the chatbot will draw from when presenting results to the user.

Step 8 — Database → Dynamic Charts & AI Report The database now holds everything worth showing the user. The reporting module retrieves this combined dataset and translates it into something visually meaningful — allocation charts that show where money should go, trend lines that show how relevant stocks have moved, and sentiment graphs that show the market's emotional temperature. Alongside the visuals, a written report is generated that stitches all these findings into a coherent narrative, giving the user both the picture and the explanation behind it.

Step 9 — Database → AI Financial Chatbot Not every user will be satisfied reading a report — some want to ask questions. The AI Financial Chatbot exists precisely for that. It taps into the same database to retrieve the user's profile, the strategy recommendation, and the sentiment data, then stands ready to respond in natural language. Why was this strategy chosen? Is the market looking risky right now? Should someone my age be investing aggressively? The chatbot handles these questions conversationally, turning a static output into a two-way dialogue that feels genuinely helpful rather than mechanical.

PROGRAM CODE(KEY SNIPPETS)

1. Recommendation Engine with Prolog Fallback (advisor.py)

```
def get_recommendation(age, income, goal):  
    if PROLOG_AVAILABLE:  
        query = f'recommend({age}, {income}, {goal}, Title, Strategy)'  
        result = list(prolog.query(query))  
        if result: return {'title': result[0]['Title'], ...}  
    # Python fallback logic  
    if goal == 'retirement' and age > 45 or income < 30000:  
        risk = 'low'  
    elif goal == 'wealth' and age <= 35 and income > 60000:  
        risk = 'high'  
    else: risk = 'medium'
```

2. Market Trends with Caching (app.py)

```
@st.cache_data(ttl=600)  
def fetch_trends():  
    return get_market_trends() # yfinance 5-day close prices
```

3. SQLite Data Persistence (database.py)

```
def save_user(age, income, goal, risk, title, strategy):  
    conn = sqlite3.connect('investments.db')  
    cursor.execute("INSERT INTO users (...) VALUES (?, ?, ?, ?, ?, ?)",  
        (age, income, goal, risk, title, strategy, datetime.now()))  
    conn.commit()
```

4. Keyword Chatbot (chatbot.py)

```
def get_chatbot_response(user_input):  
    if 'crypto' in user_input:  
        return 'Crypto should be max 5-10% of a high-risk portfolio.'  
    elif 'risk' in user_input:  
        return 'Risk appetite depends on your age and financial goals.'
```

OUTPUT / OBSERVATION

- Advisor Page

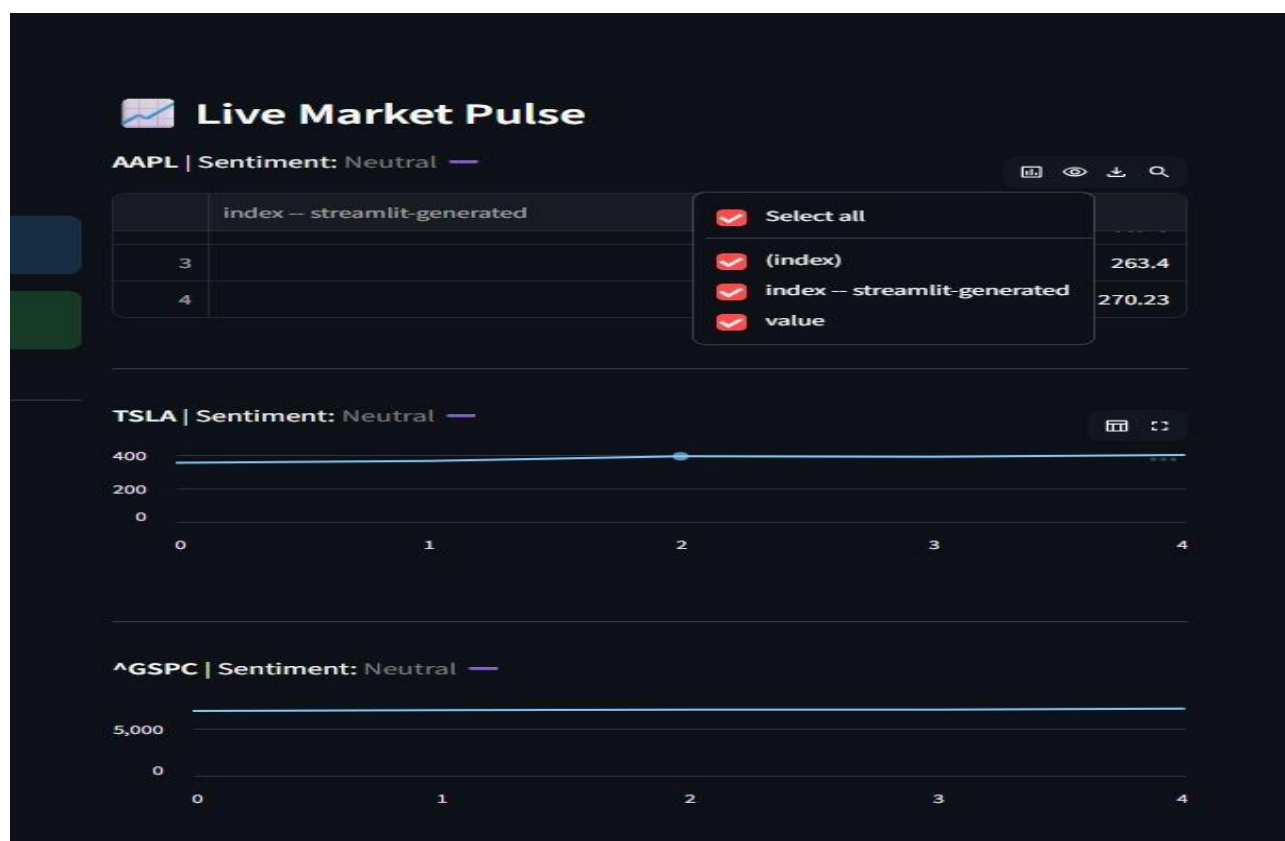
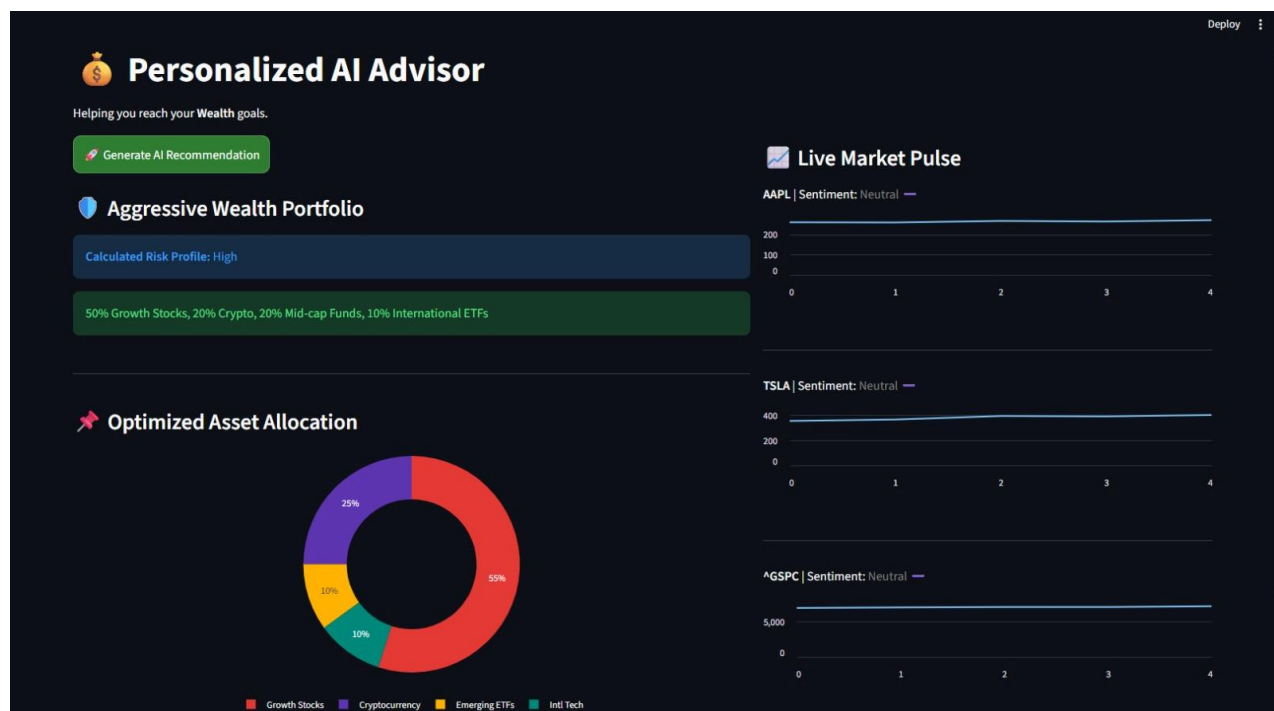
The Advisor Page is where users build their financial profile using four sidebar inputs — an age slider (18–80), a monthly income field in INR, a risk appetite selector (Low, Medium, High), and an investment goal dropdown (Wealth, Retirement, Education, Safety). Once the user clicks Generate AI Recommendation, the system returns a strategy title along with a detailed asset allocation breakdown, visualised instantly as a Plotly donut chart. A live market pulse section beneath the recommendation displays five-day price line charts for AAPL, TSLA, and the S&P 500, each accompanied by a sentiment indicator showing current market direction.

- History Page

The History Page maintains a complete record of every session. A structured table retrieves all past recommendations from the SQLite database, displaying the profile inputs, assigned strategy, and timestamp for each entry. A bar chart alongside the table shows how investment goals have been distributed across all recorded sessions, giving users a clear picture of their advisory history over time.

- Expert Chat Page

The Expert Chat Page offers a conversational interface where users can ask financial questions freely. The chatbot identifies keywords — invest, risk, stock, safe, crypto, nifty, and index — and responds with relevant, context-specific guidance based on what it detects. The full conversation is preserved within the session using `st.session_state`, so the dialogue builds continuously and users can scroll back through earlier exchanges without losing context.



LIMITATIONS AND FUTURE WORK

Current Limitations

- The system currently depends on TextBlob for sentiment analysis, which handles general language reasonably well but struggles with the specialised vocabulary and layered meaning found in financial news — producing scores that may not fully capture the true market signal embedded in complex headlines.
- The investment rules powering the Expert System are written manually in Prolog, meaning every time market regulations shift, new financial products emerge, or risk thresholds change, a developer must step in and rewrite the logic by hand — a process that does not scale well over time.
- SQLite serves the project adequately at prototype scale, but it was not built for concurrent multi-user access or large datasets; moving toward a production environment would demand a more capable database system such as PostgreSQL to handle the load reliably.
- The platform currently stops at advice — it has no connection to any live portfolio or brokerage system, so users must manually act on whatever recommendations the system produces, with no way to track whether their holdings are staying aligned with the suggested strategy.

Proposed Future Enhancements

- Replacing TextBlob with FinBERT — a language model trained specifically on financial texts — would bring domain-aware sentiment scoring to the system, correctly interpreting the jargon, context, and subtle signals that general NLP tools routinely miss.
- Adding predictive models such as LSTM networks or Random Forest classifiers would give the system a forward-looking capability, enabling it to forecast likely price movements based on historical patterns rather than only describing where the market currently stands.

- Introducing user authentication and persistent session management would allow individuals to log back in, review earlier recommendations, track portfolio performance over time, and update their profile as their financial circumstances evolve.
- Connecting to brokerage APIs such as Zerodha Kite or Alpaca would close the gap between recommendation and execution, allowing the system to place trades and rebalance portfolios directly within the same platform that generated the advice.
- Deploying the system on cloud infrastructure such as AWS or GCP would make it accessible to many users simultaneously, ensure market data stays continuously refreshed, and provide the security and uptime standards that a financial application requires.
- Upgrading the chatbot with a large language model backend would allow it to reason dynamically across a user's full financial picture in real time, supporting richer multi-turn conversations and more nuanced, context-aware responses than a database-query approach can offer.

CONCLUSION

Building a system that genuinely assists people in making better financial decisions is not a trivial undertaking. It requires not just technical competence but a thoughtful architecture — one where every component plays a defined role and the pieces fit together without friction. This project achieves exactly that. Each subsystem, from the Prolog rule engine to the TextBlob sentiment scorer, was chosen deliberately, and the way they communicate through the Advisor Bridge and SQLite database reflects a design philosophy that prioritises clarity, traceability, and reliability over unnecessary complexity.

What makes this system stand apart from a simple stock screener or a generic financial calculator is the layering of different forms of intelligence. Rule-based reasoning brings explainability — every strategy recommendation can be traced back to a specific logical condition, which means users are never left wondering why they received a particular piece of advice.

Real-time market data brings relevance — recommendations respond to what the market is actually doing today, not what it was doing when the system was first built. Sentiment analysis brings awareness — the emotional temperature of financial news is factored in alongside hard price figures, giving the system a more complete picture of the investment environment.

Together, these three dimensions produce guidance that is simultaneously grounded, current, and contextually aware. Taken together, this project makes a clear case that intelligent, personalised financial advisory does not have to be expensive, inaccessible, or opaque. With the right architecture and the right combination of technologies, it can be transparent, responsive, and available to anyone who has a financial goal worth working toward.

REFERENCES

- Lam, M. (2004). Neural network techniques for financial performance prediction: Integrating fundamental and technical analysis. *Decision Support Systems*, 37(4), 567–581.
- Jiang, Z., Xu, D., & Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*.
- Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1), 1–8.
- Loria, S. (2020). *TextBlob Documentation: Simplified Text Processing*. Retrieved from <https://textblob.readthedocs.io>
- Arner, D. W., Barberis, J., & Buckley, R. P. (2016). FinTech, RegTech, and the reconceptualization of financial regulation. *Northwestern Journal of International Law & Business*, 37(3), 371–413.
- Merritt, R. (2012). *Programming in Prolog: Using the ISO Standard*. Springer Science & Business Media.